Recurrent neural networks

Contents

6.1 Recurrent Neural Networks (RNNs) **1**
6.1.1 Basic RNN . . . 2
6.1.1.1 The tunable parameters of a basic RNN . . . 2
6.1.1.2 Measure the activities of the neurons in a basic RNN . . . 4
6.1.2 Training RNNs . . . 5
6.1.2.1 Using the chain rule . . . 5
6.1.2.2 Using Backpropagation through time (BPTT) . . . 5

6.2 Vanishing and exploding of gradients **6**
6.2.1 Vanishing gradients lead to exponential forgetting . . . 6

6.3 Echo-state networks **7**
6.3.1 Initialization of weight matrices . . . 7

6.4 From RNN to LSTM **8**
6.4.1 Complexity . . . 10

6.5 References **11**

6.1 Recurrent Neural Networks (RNNs)

- Feedforward networks:

  Data:
  $$\left\{ \left(\underline{\mathbf{x}}^{(\alpha)}, \underline{\mathbf{y}}^{(\alpha)}_{\text{True}} \right) \right\}, \quad \alpha = 1, \ldots, p$$

  - any two sample pairs $\left(\underline{\mathbf{x}}^{(\alpha)}, \underline{\mathbf{y}}^{(\alpha)}_{\text{True}} \right)$ and $\left(\underline{\mathbf{x}}^{(\alpha')}, \underline{\mathbf{y}}^{(\alpha')}_{\text{True}} \right)$ are assumed to be i.i.d.
  - Their respective predictions $\underline{\mathbf{y}}(\underline{\mathbf{x}}^{(\alpha)}; \underline{\mathbf{w}})$ and $\underline{\mathbf{y}}(\underline{\mathbf{x}}^{(\alpha')}; \underline{\mathbf{w}})$ are also independent from one another.

- RNN $\widehat{=}$ Neural network with loops.

  Data:
  $$\left\{ \left(\underline{\mathbf{x}}^{(\alpha,t)}, \underline{\mathbf{y}}^{(\alpha,t)}_{\text{True}} \right)_{t=1}^{n_\alpha} \right\}, \quad \alpha = 1, \ldots, p$$

  - p i.i.d.sequences
  - each sequence α contains n_α time steps
  - Use information from the previous step to predict the next step.

6.1.1 Basic RNN

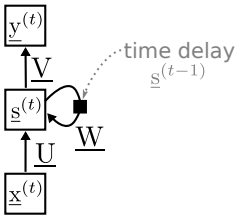


Figure 6.1.1: Basic RNN architecture

6.1.1.1 The tunable parameters of a basic RNN

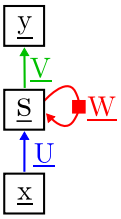


Figure 6.1.2: Basic RNN architecture

Let

N be the number of input variables $\rightarrow \underline{\mathbf{x}}^{(t)} \in \mathbb{R}^N$

H be the number of hidden nodes

M be the number of output nodes

Identify the free parameters (trainable weights) of the network:

- input-to-hidden: $\underline{\mathbf{U}} = (\underline{\mathbf{u}}_1, \ldots, \underline{\mathbf{u}}_i, \ldots, \underline{\mathbf{u}}_H) \in \mathbb{R}^{H,N}$
- hidden-to-hidden: $\underline{\mathbf{W}} = (\underline{\mathbf{w}}_1, \ldots, \underline{\mathbf{w}}_j, \ldots, \underline{\mathbf{u}}_H) \in \mathbb{R}^{H,H}$
- hidden-to-output: $\underline{\mathbf{V}} = (\underline{\mathbf{v}}_1, \ldots, \underline{\mathbf{v}}_k, \ldots, \underline{\mathbf{v}}_H) \in \mathbb{R}^{M,H}$

Remark: The weight matrices are constructed slightly differently than how we defined them for MLPs. We now stack **row** vectors instead of **column** vectors. Reason: brevity, to avoid putting transpose everywhere

- input-to-hidden:

  The weights from each input x_j to each hidden node h_i:

$$\underline{\mathbf{u}}_i = (u_{i1}, \ldots, u_{ij}, \ldots, u_{iN}) \text{ with } j = 1, \ldots, N \text{ and } i = 1, \ldots, H \tag{6.1.1}$$

$$\underline{\mathbf{U}} = \begin{pmatrix} — & \underline{\mathbf{u}}_1 & — \\ — & \underline{\mathbf{u}}_2 & — \\ & \vdots & \\ — & \underline{\mathbf{u}}_H & — \end{pmatrix} \in \mathbb{R}^{H,N} \tag{6.1.2}$$

- hidden-to-hidden:

  The weights to each hidden node h_i:

$$\underline{\mathbf{w}}_i = (w_{i1}, \ldots, w_{ij}, \ldots, w_{iH}) \text{ with } i, j = 1, \ldots, H \tag{6.1.3}$$

$$\underline{\mathbf{W}} = \begin{pmatrix} — & \underline{\mathbf{w}}_1 & — \\ — & \underline{\mathbf{w}}_2 & — \\ & \vdots & \\ — & \underline{\mathbf{w}}_H & — \end{pmatrix} \in \mathbb{R}^{H,H} \tag{6.1.4}$$

- hidden-to-output:

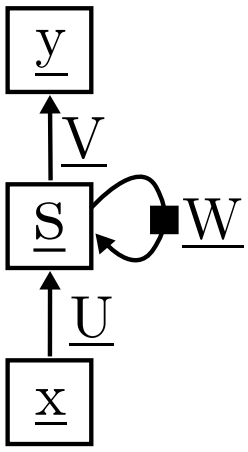
  The weights to each output node y_k:

$$\underline{\mathbf{v}}_k = (\mathrm{v}_{k1}, \ldots, \mathrm{v}_{kj}, \ldots, \mathrm{v}_{kH}) \text{ with } k = 1, \ldots, M \tag{6.1.5}$$

$$\underline{\mathbf{V}} = \begin{pmatrix} — & \underline{\mathbf{v}}_1 & — \\ — & \underline{\mathbf{v}}_2 & — \\ & \vdots & \\ — & \underline{\mathbf{v}}_M & — \end{pmatrix} \in \mathbb{R}^{M,H} \tag{6.1.6}$$

6.1.1.2 Measure the activities of the neurons in a basic RNN

First we define $s_i^{(t)}$ and $y_k^{(t)}$, then we switch from component-wise notation to vector notation



$$
\begin{aligned}
s_i^{(t)} &= \tanh\left(\sum_{k=1}^{N} U_{ik}\, x_k^{(t)} + \sum_{j=1}^{H} W_{ij}\, s_j^{(t-1)} + b_i^{\mathrm{s}}\right), \\
y_k^{(t)} &= f\left(\sum_{j=1}^{H} V_{kj}\, s_j^{(t)} + b_k^{\mathrm{y}}\right),
\end{aligned}
$$

$$
\begin{aligned}
\text{e.g. } f(\cdot) &:= \text{e.g. } \mathrm{softmax}(\cdot) \text{ for classification} \\
\underline{\mathbf{s}}^{(t)} &= \tanh\left(\underline{\mathbf{U}}\,\underline{\mathbf{x}}^{(t)} + \underline{\mathbf{W}}\,\underline{\mathbf{s}}^{(t-1)} + \underline{\mathbf{b}}^{\mathrm{s}}\right), \\
\underline{\mathbf{y}}^{(t)} &= f\left(\underline{\mathbf{V}}\, s^{(t)} + \underline{\mathbf{b}}^{\mathrm{y}}\right),
\end{aligned}
$$

$$
\text{e.g. } f(\cdot) := \text{e.g. } \mathrm{softmax}(\cdot) \text{ for classification}
$$

6.1.2 Training RNNs

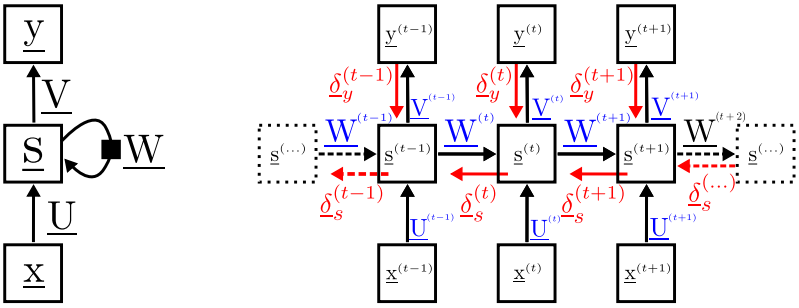
6.1.2.1 Using the chain rule

- cost function $e^{(\alpha,t)}$ for time step t of training sequence α

$$E^T = \frac{1}{p}\sum_{\alpha=1}^{p}\frac{1}{n_\alpha}\sum_{t=1}^{n_\alpha} e^{(\alpha,t)}\,, \quad \text{for training set} \quad \left\{\{\underline{\mathbf{x}}^{(\alpha,t)}, \underline{\mathbf{y}}^{(\alpha,t)}\}_{t=1}^{n_\alpha}\right\}_{\alpha=1}^{p}$$

- gradient computation using the chain rule
- computational complexity: $\mathcal{O}(H^2)$, i.e. quadratic in the number of recurrent weights

6.1.2.2 Using Backpropagation through time (BPTT)



- assume all unfolded weights, e.g. $\underline{\mathbf{W}}^{(t)}$, are independent
- compute gradients of deep MLP with backpropagation ($\mathcal{O}(n)$, where n is the total number of all weights in the unrolled network)
- average all computed gradients, i.e. $\frac{\partial e^{(\alpha,t)}}{\partial \underline{\mathbf{W}}^{(\alpha,\tau)}}$, for weight update:

$$\Delta\underline{\mathbf{W}} = -\eta\frac{\partial E^T}{\partial \underline{\mathbf{W}}} = -\frac{\eta}{p}\sum_{\alpha=1}^{p}\frac{1}{n_\alpha}\sum_{t=1}^{n_\alpha}\frac{\partial e^{(\alpha,t)}}{\partial \underline{\mathbf{W}}} \approx -\frac{\eta}{p}\sum_{\alpha=1}^{p}\frac{1}{n_\alpha}\sum_{t=1}^{n_\alpha}\frac{1}{t}\sum_{\tau=1}^{t}\frac{\partial e^{(\alpha,t)}}{\partial \underline{\mathbf{W}}^{(\alpha,\tau)}}$$

The backpropagation algorithm provides an *exact* computation of the gradients. However, since BPPT involves unfolding the weights (unrolling the network), this leads to an *approximation of the gradients*.

(for details see Williams and Zipser, 1995)

6.2 Vanishing and exploding of gradients

A simple RNN learns from short-term errors faster (exponentially) than from errors due to longer ($\Delta t \gg 1$) distances in a sequence.

Example:Estimating the topic of a lengthy text where the "clue" is mentioned in the very beginning.

The gradient vanishes.

6.2.1 Vanishing gradients lead to exponential forgetting

- an RNN with no inputs and a linear transfer function

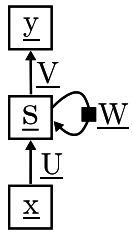
$$\underline{\mathbf{s}}^{(t)} \quad = \quad \underline{\mathbf{W}}\,\underline{\mathbf{s}}^{(t-1)} \quad = \quad (\underline{\mathbf{W}})^t\,\underline{\mathbf{s}}^{(0)}$$

- eigenvalue decomposition: $\underline{\mathbf{W}} = \underline{\mathbf{U}}\,\underline{\mathbf{\Lambda}}\,\underline{\mathbf{U}}^\top$

$$\underline{\mathbf{s}}^{(t)} \quad = \quad \underbrace{\underline{\mathbf{U}}}_{\text{rotate back}}\,\overbrace{(\underline{\mathbf{\Lambda}})^t}^{\text{exponential scaling}}\,\underbrace{\underline{\mathbf{U}}^\top\underline{\mathbf{s}}^{(0)}}_{\text{rotation}}$$

- long-term activity dominated by the largest $|\lambda_1|$
  - $|\lambda_1| > 1 \quad \rightsquigarrow \quad \lim\limits_{t\to\infty} s_i^{(t)} = \pm\infty\,, \forall i$ (exploding activity)
  - $|\lambda_1| < 1 \quad \rightsquigarrow \quad \lim\limits_{t\to\infty} s_i^{(t)} = 0\,, \forall i$ (vanishing activity)
- local error terms in backpropagation through time
- over many time steps the local errors may also vanish
- small short-term errors overshadow large long-term errors

6.3 Echo-state networks



- system dynamics determine the input representation
  - non-linear preprocessing step with memory
- address vanishing gradients by keeping $\underline{\mathbf{W}}$ and $\underline{\mathbf{U}}$ fixed
  - output weights $\underline{\mathbf{V}}$ can be trained by e.g. linear regression
  - hidden "reservoir" maintains inputs/errors over time
- system dynamics determine how fast inputs/errors are forgotten

6.3.1 Initialization of weight matrices

- eigenvalues λ_i of the system's *Jacobian* $\underline{\mathbf{J}}$

$$J_{ij}^{(t)} = \frac{\partial h_i^{(t)}}{\partial h_j^{(t-1)}} = W_{ij}\, f'(h_j^{(t)})$$

$$h_i^{(t)} = \sum_{j=1}^{H} W_{ij}\, f(h_j^{(t-1)})$$

- eigenvectors with $|\lambda_i| \approx 1$ vanish/explode *very slowly*
  - inputs/errors are maintained for a long time
- generate random $\underline{\mathbf{W}}$, and $\underline{\mathbf{U}}$ where all $|\lambda_j| \approx r$
  - $f'(h_j^{(t)}) < 1$ is a contraction $\rightsquigarrow$ $r > 1$
  - values between $r = 1.2$ and $r = 3$ are used in practice

(Jaeger and Haas, 2004; Goodfellow et al., 2016, Section 10.8)
(Jaeger and Haas, 2004; Goodfellow et al., 2016, Section 10.8)

6.4 From RNN to LSTM

The long short-term memory (LSTM) cell is an extension of the basic RNN architecture with memory state cells.

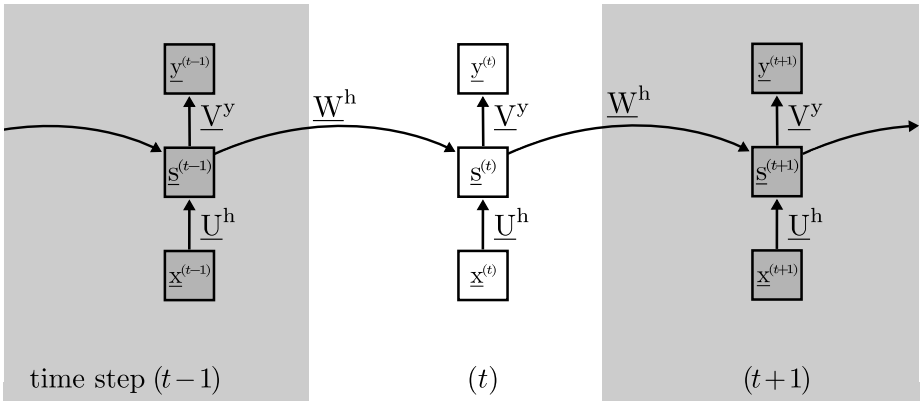


Figure 6.4.1: We start constructing our LSTM from a simple RNN unrolled in time.

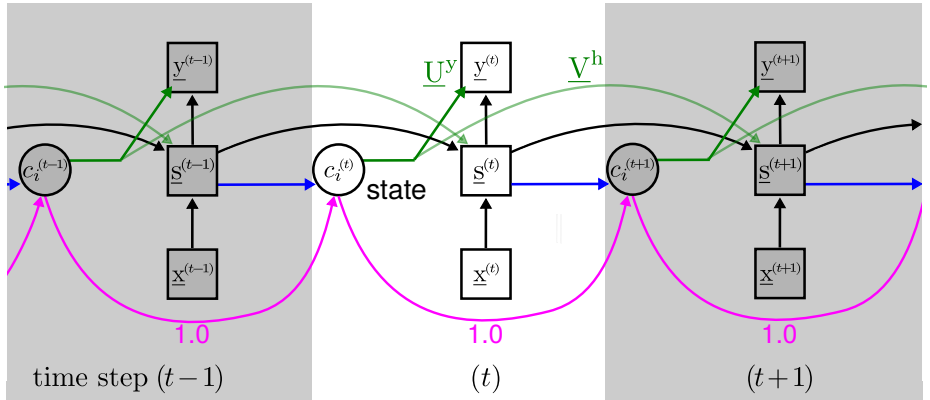
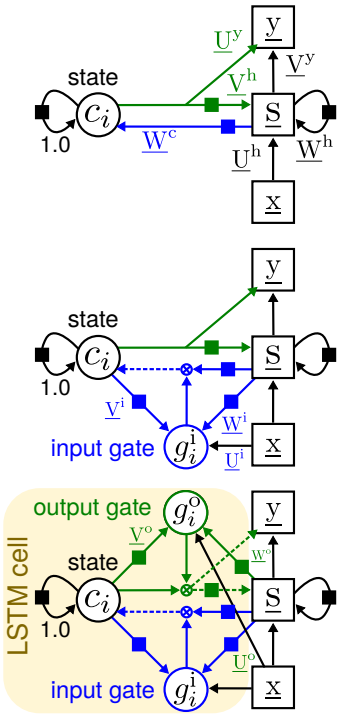


Figure 6.4.2: Adding memory state cells to the simple RNN unrolled in time.

- extend the simple RNN with state nodes $\underline{\mathbf{c}}$
- leaky unit c_i with $\alpha_i = 1$
- time delayed feedback to hidden layer
- time delayed storage into state layer

- introducing an input ("write") gate g_i^{i}
  - gate transfer function $\sigma(x) = \left(1 + e^{-x}\right)^{-1}$
- state input is multiplied ($\otimes$) with gate
- state c_i only changes when $g_i^{\mathrm{i}(t)} \gg 0$
- introducing an output ("read") gate g_i^{o}
  - gate transfer function $\sigma(x) = \left(1 + e^{-x}\right)^{-1}$
- state output is multiplied ($\otimes$) with gate
- error $\delta_i^{(t)}$ only changes when $g_i^{\mathrm{o}(t)} \gg 0$
- both gates and the state form an LSTM cell

- gates determine access to the state c_i
  - g_i^{i} learns what to remember
  - g_i^{o} learns when use the memory
- gates regulate the flow of activity and error
  - g_i^{i} regulates the forward-pass
  - g_i^{o} regulates the backward-pass



$$
\begin{aligned}
\underline{\mathbf{c}}^{(t)} &= \underline{\mathbf{c}}^{(t-1)} + \underline{\mathbf{g}}^{\mathrm{i}(t)} \odot \tanh\left(\underline{\mathbf{W}}^{\mathrm{c}} \underline{\mathbf{s}}^{(t-1)}\right) \\
\underline{\mathbf{y}}^{(t)} &= f\left(\underline{\mathbf{V}}^{\mathrm{y}}\, \underline{\mathbf{s}}^{(t)} + \underline{\mathbf{b}}^{\mathrm{y}} + \underline{\mathbf{U}}^{\mathrm{y}} \left(\underline{\mathbf{c}}^{(t)} \odot \underline{\mathbf{g}}^{\mathrm{o}(t)}\right)\right), \qquad \text{e.g. } f(\cdot) = \mathrm{softmax}(\cdot) \\
\underline{\mathbf{s}}^{(t)} &= \tanh\left(\underline{\mathbf{U}}^{\mathrm{h}}\, \underline{\mathbf{x}}^{(t)} + \underline{\mathbf{W}}^{\mathrm{h}}\, \underline{\mathbf{s}}^{(t-1)} + \underline{\mathbf{b}}^{\mathrm{s}} + \underline{\mathbf{V}}^{\mathrm{h}} \left(\underline{\mathbf{c}}^{(t-1)} \odot \underline{\mathbf{g}}^{\mathrm{o}(t)}\right)\right) \\
\underline{\mathbf{g}}^{\mathrm{i}(t)} &= \sigma\left(\underline{\mathbf{U}}^{\mathrm{i}}\, \underline{\mathbf{x}}^{(t)} + \underline{\mathbf{V}}^{\mathrm{i}}\, \underline{\mathbf{c}}^{(t-1)} + \underline{\mathbf{W}}^{\mathrm{i}}\, \underline{\mathbf{s}}^{(t-1)} + \underline{\mathbf{b}}^{\mathrm{i}}\right) \\
\underline{\mathbf{g}}^{\mathrm{o}(t)} &= \sigma\left(\underline{\mathbf{U}}^{\mathrm{o}}\, \underline{\mathbf{x}}^{(t)} + \underline{\mathbf{V}}^{\mathrm{o}}\, \underline{\mathbf{c}}^{(t-1)} + \underline{\mathbf{W}}^{\mathrm{o}}\, \underline{\mathbf{s}}^{(t-1)} + \underline{\mathbf{b}}^{\mathrm{o}}\right)
\end{aligned}
$$

Q 6.4.1: *Why does* $\tanh(\cdot)$ *make sense here?*

(Hochreiter and Schmidhuber, 1997; Goodfellow et al., 2016, Section 10.10)

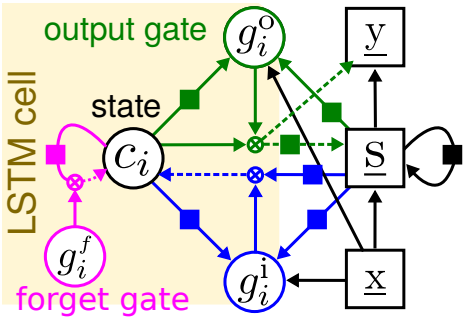


Figure 6.4.3: Basic RNN architecture

$$\underline{\mathbf{g}}_i^{\mathrm{f}(t)} = \sigma\left(\underline{\mathbf{U}}^{\mathrm{f}}\,\underline{\mathbf{x}}^{(t)} + \underline{\mathbf{V}}^{\mathrm{f}}\,\underline{\mathbf{c}}^{(t-1)} + \underline{\mathbf{W}}^{\mathrm{f}}\,\underline{\mathbf{s}}^{(t-1)} + \underline{\mathbf{b}}^{\mathrm{f}}\right) \tag{6.4.1}$$

Consequently:

$$\underline{\mathbf{c}}^{(t)} = \underline{\mathbf{g}}^{\mathrm{f}(t)} \odot \underline{\mathbf{c}}^{(t-1)} + \underline{\mathbf{g}}^{\mathrm{i}(t)} \odot \tanh\left(\underline{\mathbf{W}}^{\mathrm{c}}\underline{\mathbf{s}}^{(t-1)}\right) \tag{6.4.2}$$

6.4.1 Complexity

For $\underline{\mathbf{x}} \in \mathbb{R}^N, \underline{\mathbf{s}} \in \mathbb{R}^H, \underline{\mathbf{y}} \in \mathbb{R}^M$:

Simple RNN:

- $\underline{\mathbf{U}}^{\mathrm{h}} \in \mathbb{R}^{H,N}, \quad \underline{\mathbf{b}}^{\mathrm{h}} \in \mathbb{R}^H$
- $\underline{\mathbf{W}}^{\mathrm{h}} \in \mathbb{R}^{H,H}$
- $\underline{\mathbf{V}}^{\mathrm{y}} \in \mathbb{R}^{M,H}, \quad \underline{\mathbf{b}}^{\mathrm{y}} \in \mathbb{R}^M$

LSTM: All of the above in addition to... for $\underline{\mathbf{c}}, \underline{\mathbf{g}}^{\mathrm{i}}, \underline{\mathbf{g}}^{\mathrm{o}}, \underline{\mathbf{g}}^{\mathrm{f}} \in \mathbb{R}^K$ (often $K = H$):

- for $\underline{\mathbf{c}}$: $\underline{\mathbf{W}}^{\mathrm{c}} \in \mathbb{R}^{K,H}$, $\underline{\mathbf{U}}^{\mathrm{y}} \in \mathbb{R}^{M,K}$, $\underline{\mathbf{V}}^{\mathrm{h}} \in \mathbb{R}^{H,K}$

  for $\underline{\mathbf{g}}^{\mathrm{i}}$:

  * $\underline{\mathbf{U}}^{\mathrm{i}} \in \mathbb{R}^{K,N}$
  * $\underline{\mathbf{V}}^{\mathrm{i}} \in \mathbb{R}^{K,K}$
  * $\underline{\mathbf{W}}^{\mathrm{i}} \in \mathbb{R}^{K,H}, \quad \underline{\mathbf{b}}^{\mathrm{i}} \in \mathbb{R}^K$
- duplicate $\underline{\mathbf{U}}^{\mathrm{i}}, \underline{\mathbf{V}}^{\mathrm{i}}, \underline{\mathbf{W}}^{\mathrm{i}}$ for the output gate $\rightsquigarrow$ $\underline{\mathbf{U}}^{\mathrm{o}}, \underline{\mathbf{V}}^{\mathrm{o}}, \underline{\mathbf{W}}^{\mathrm{o}}$ and $\underline{\mathbf{b}}^{\mathrm{o}}$
- and for the forget gate $\rightsquigarrow$ $\underline{\mathbf{U}}^{\mathrm{f}}, \underline{\mathbf{V}}^{\mathrm{f}}, \underline{\mathbf{W}}^{\mathrm{f}}$ and $\underline{\mathbf{b}}^{\mathrm{f}}$

Q 6.4.2: *What about the complexity of echo state networks?*

- The hidden-output weights $\underline{\mathbf{V}}$ are the only trainable parameters in echo state networks.

6.5 References

References

Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. MIT Press, 2016. http://www.deeplearningbook.org.

Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. *Neural Computation*, 9(8):1735–1780, 1997.

Herbert Jaeger and Harald Haas. Harnessing nonlinearity: Predicting chaotic systems and saving energy in wireless communication. *Science*, 304(5667):78–80, 2004. URL http://science.sciencemag.org/content/304/5667/78.full.pdf.

Ronald J. Williams and David Zipser. Backpropagation. chapter Gradient-based Learning Algorithms for Recurrent Networks and Their Computational Complexity, pages 433–486. L. Erlbaum Associates Inc., 1995.